

# **API Testing Report**

**Name : Shahbaz Ahmad**

**GitHub branch: shahbaz\_assessment**

**Date: 21-07-2026**

## **Question 1 ()**

### **Objective**

To design, develop, and execute API automation tests ensuring reliability, scalability, and correctness of REST APIs using a structured framework. Website <https://jsonplaceholder.typicode.com/>

GithubLink:-

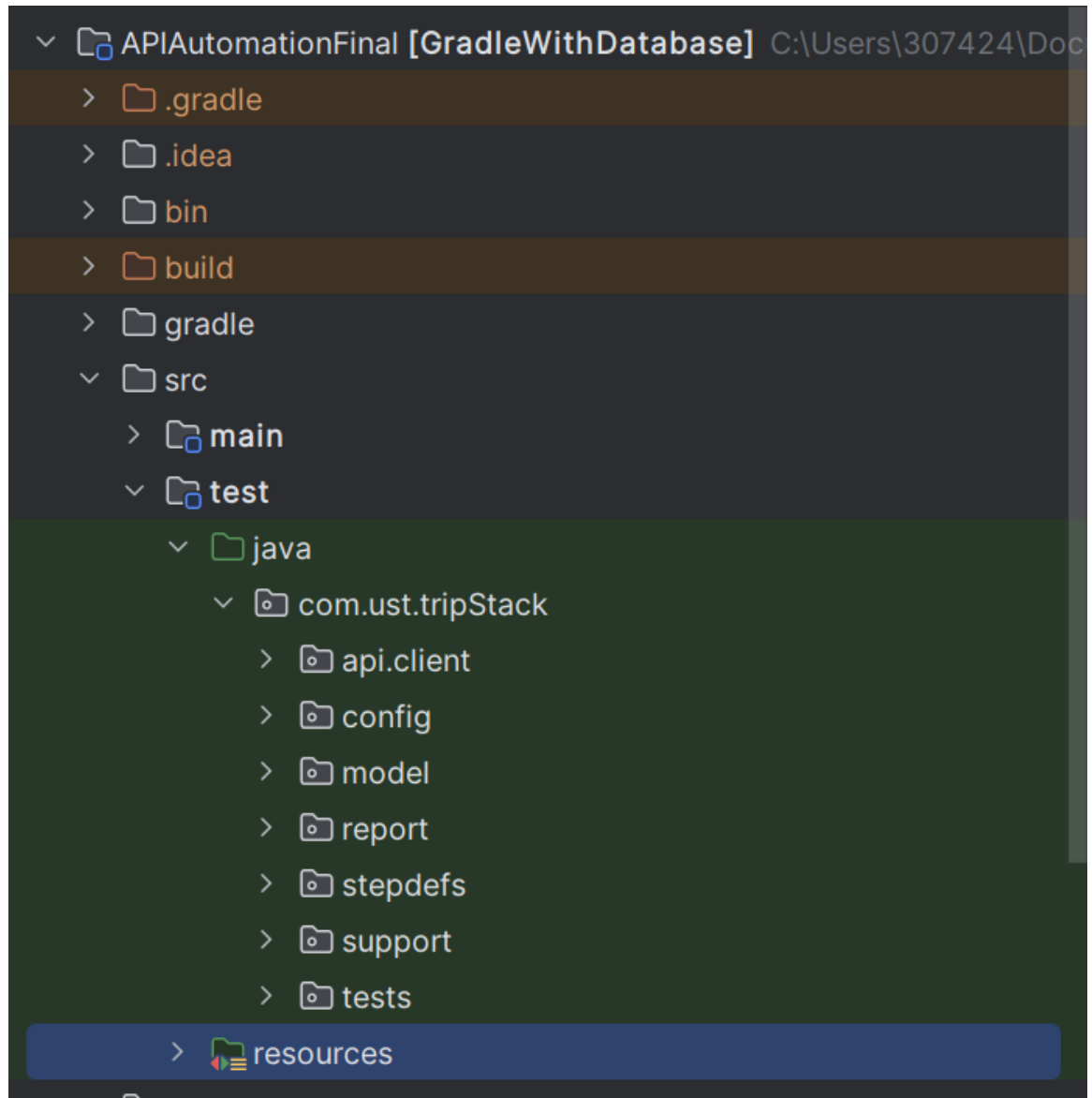
### **Environment Setup**

Java, Maven, RestAssured, IntelliJ IDEA / Vs code Allure Reporting.

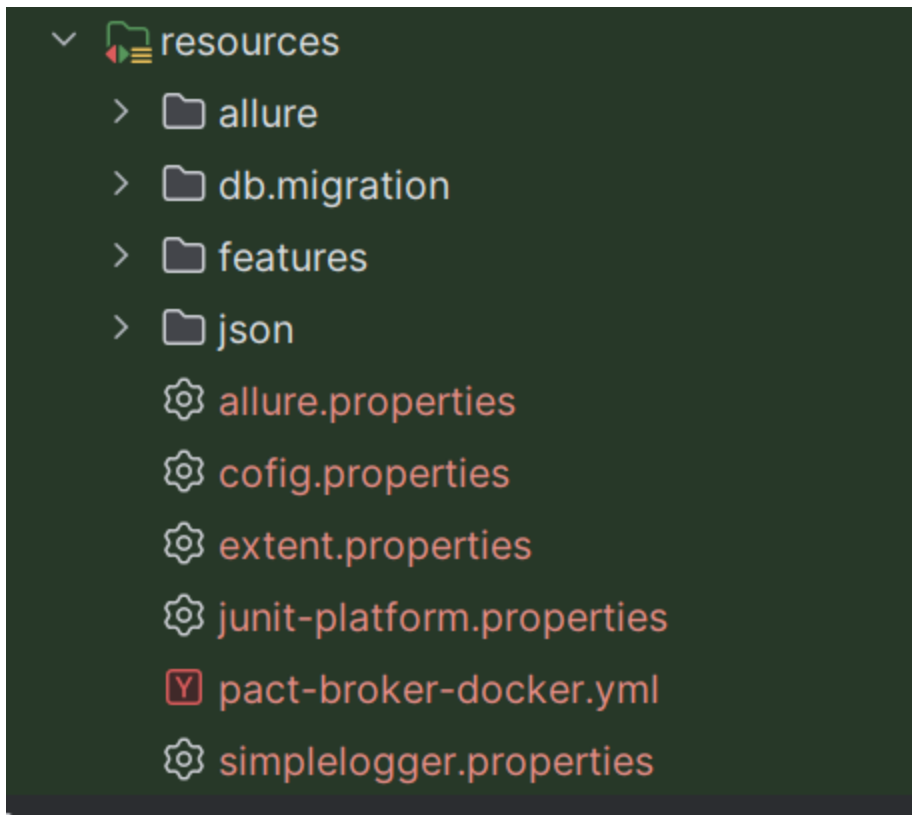
### **Workflow**

1. Read configuration from Config.java
2. Build request using SpecFactory
3. Execute post API
4. Validate response
5. Generate report

## Folder Structure



- Ⓢ HomeClient
- Ⓢ PaymentClient
- Ⓢ SeatClient
- Ⓢ TicketClient
- ▼ Ⓛ config
  - Ⓢ DatabaseConfig
- ▼ Ⓛ model
  - Ⓢ BookingRow
  - Ⓢ OrderRow
- ▼ Ⓛ report
  - Ⓢ ExtentManager
  - Ⓢ ExtentTestListener
- ▼ Ⓛ stepdefs
  - Ⓢ FlightBookingSteps
- ▼ Ⓛ support
- ▼ Ⓛ tests
  - Ⓢ BookingE2E
  - Ⓢ RunCucumberTest
  - Ⓢ WireMockTest



## **Scenario**

"As a QA Engineer, I want to create a new post and verify the response.

## **Commands**

- ./gradlew clean
- ./gradlew build
- ./gradlew test --tests "com.ust.tripStack.\*"
- allure serve target/allure-results

## **Result Evidence & Screenshots**

### **Code**

```
package com.ust.tripStack.tests;
```

```
import com.ust.tripStack.report.ExtentTestListener;  
import io.qameta.allure.Epic;
```

```
import io.qameta.allure.Feature;
import io.qameta.allure.Owner;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
```

```
import static com.github.tomakehurst.wiremock.client.WireMock.*;
import static com.github.tomakehurst.wiremock.stubbing.Scenario.STARTED;
import static io.restassured.RestAssured.given;
import static java.net.http.HttpResponse.BodyHandlers.ofString;
import static org.hamcrest.Matchers.equalTo;
```

```
import java.io.IOException;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpTimeoutException;
import java.time.Duration;
```

```
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.junit.jupiter.api.extension.RegisterExtension;
```

```
import com.github.tomakehurst.wiremock.junit5.WireMockExtension;
import static com.github.tomakehurst.wiremock.core.WireMockConfiguration.wireMockConfig;
import static org.junit.jupiter.api.Assertions.*;
import io.restassured.RestAssured;
```

```
@Epic("tripStack Journeys")
@Feature("Full Checkout Journey")
@Owner("Shahbaz Ahmad")
@ExtendWith(ExtentTestListener.class)
public class WireMockTest {
```

```
    @RegisterExtension
    static WireMockExtension vm = WireMockExtension.newInstance()
        .options(wireMockConfig().dynamicPort())
        .build();
```

```
    private HttpClient client;
```

```
    @BeforeEach
    void setup(){
```

```

    RestAssured.baseURI = vm.baseUrl();
    client = HttpClient.newHttpClient();
}

@AfterEach
void cleanup() {
    vm.resetAll();
}

@Test
@DisplayName("mock payment decline")
void paymentDecline(){
    vm.stubFor(get(urlEqualTo("/mocks.tripstack.doomple.com/payment"))
        .willReturn(aResponse()
            .withStatus(402)
            .withBody("{"latency":"500","timeout":"5000","reset":"true"}")
            .withHeader("Content-Type", "application/json")
        ));

    given()
        .when()
        .get("/mocks.tripstack.doomple.com/payment")
        .then()
        .statusCode(402)
        .body("timeout", equalTo("5000"));

    vm.verify(getRequestedFor(urlEqualTo("/mocks.tripstack.doomple.com/payment")));
}

@Test
@DisplayName("seat expiry conflict")
void seatHoldConflict(){
    vm.stubFor(get(urlEqualTo("/mocks.tripstack.doomple.com/seathold"))
        .willReturn(aResponse()
            .withStatus(409)
            .withBody("{"expiry-period":"10000"}")
            .withHeader("Content-Type", "application/json")
        ));

    given()
        .when()
        .get("/mocks.tripstack.doomple.com/seathold")
        .then()
        .statusCode(409)

```

```

        .body("expiry-period", equalTo("10000"));

        vm.verify(getRequestedFor(urlEqualTo("/mocks.tripstack.doomple.com/seathold")));
    }
}

```

## StepDefiniton

```
package com.ust.tripStack.stepdefs;
```

```

import com.ust.tripStack.api.client.*;
import com.ust.tripStack.model.BookingRow;
import com.ust.tripStack.config.DatabaseConfig;
import com.ust.tripStack.report.ExtentTestListener;
import com.ust.tripStack.support.DbSupport;
import com.ust.tripStack.support.TestEnvironment;
import io.cucumber.java.en.Given;
import io.cucumber.java.en.Then;
import io.cucumber.java.en.When;
import io.qameta.allure.Epic;
import io.qameta.allure.Feature;
import io.qameta.allure.Owner;
import io.restassured.response.Response;
import org.junit.jupiter.api.extension.ExtendWith;

```

```

import static io.restassured.module.jsv.JsonSchemaValidator.matchesJsonSchemaInClasspath;
import static org.junit.jupiter.api.Assertions.*;

```

```

@Epic("tripStack Journeys")
@Feature("Full Checkout Journey")
@Owner("Shahbaz Ahmad")
@ExtendWith(ExtentTestListener.class)
public class FlightBookingSteps {

```

```

    String user = TestEnvironment.required("DB_USER");
    String db_password = TestEnvironment.required("DB_PASSWORD");
    String databaseurl = TestEnvironment.required("DB_JDBC_URL");

```

```

    AuthClient auth = new AuthClient();
    HomeClient home = new HomeClient();
    SeatClient seat = new SeatClient();
    PaymentClient payment = new PaymentClient();
    TicketClient ticketClient = new TicketClient();
    DbSupport dbSupport = new DbSupport(new DatabaseConfig(databaseurl, user, db_password));

```

```

    String email = TestEnvironment.required("CUSTOMER_MAIL");
    String password = TestEnvironment.required("CUSTOMER_PASSWORD");

```

```

static String token;
static String bookingId;
static String pnr;

@Given("customer logs into tripStack")
public void customer_logs_into_tripStack() {
    Response res = auth.login(email, password);
    token = res.jsonPath().getString("token");
    assertNotNull(token, "Token should not be null after login");
}

@Given("customer resets and searches for flight")
public void customer_resets_and_searches_for_flight() {
    home.reset(token);
    auth.authMe(token).then().statusCode(200);

    Response flight = home.searchFlight(token, "BLR", "CCU")
        .then().extract().response();
    assertEquals(200, flight.statusCode(), "Flight search failed");
}

@When("customer selects seat using API")
public void customer_selects_seat_using_api() {
    Response seatRes = seat.getFlight(token, "FL-BLRCCU-51");
    assertEquals(200, seatRes.statusCode(), "Seat fetch failed");
}

@When("customer books seat using API")
public void customer_books_seat_using_api() {
    Response seatBooking = seat.booking(token);
    assertEquals(201, seatBooking.statusCode(), "Seat booking failed");

    bookingId = seatBooking.jsonPath().getString("id");
    String state = seatBooking.jsonPath().getString("state");
    int amountPaise = seatBooking.jsonPath().getInt("amountPaise");

    assertNotNull(bookingId, "Booking id should not be null");
    assertEquals("HELD", state, "Newly created booking should start in HELD state");
    assertTrue(amountPaise > 0, "amountPaise should be a positive value");
}

@Then("booking should be visible in order")
public void booking_should_be_visible_in_order() {
    assertNotNull(bookingId, "Booking id should be present after booking");
}

@When("customer pays and confirms booking")
public void customer_pays_and_confirms_booking() {

```



```

Response paymentResponse = payment.payment(token, bookingId);
assertTrue(paymentResponse.statusCode() == 200 || paymentResponse.statusCode() == 201,
    "Payment initiation failed with status " + paymentResponse.statusCode());

Response confirmRes = payment.confirm(token, bookingId);
assertTrue(confirmRes.statusCode() == 200 || confirmRes.statusCode() == 201,
    "Payment confirmation failed with status " + confirmRes.statusCode());
}

@Then("booking should be visible in ticket API")
public void booking_should_be_visible_in_ticket_api() {
    Response getBooking = ticketClient.getBooking(token);
    assertEquals(200, getBooking.statusCode(), "Get booking failed");

    System.out.println(getBooking.jsonPath().prettyPrint());

    // Response root is a JSON array -> always index into [0]
    pnr = getBooking.jsonPath().getString("[0].pnr");
    String state = getBooking.jsonPath().getString("[0].state");
    String idFromList = getBooking.jsonPath().getString("[0].id");

    assertNotNull(pnr, "PNR should not be null");
    assertFalse(pnr.startsWith "["), "PNR extraction bug: got array string instead of value — " + pnr);
    assertTrue(pnr.matches("TS-\\d+-\\d+"), "PNR format unexpected: " + pnr);
    assertEquals("CONFIRMED", state, "Booking state should be CONFIRMED after payment confirmation");
    assertEquals(bookingId, idFromList, "Booking id mismatch between booking step and ticket API");
}

@Then("booking should be stored in database")
public void booking_should_be_stored_in_database() {
    BookingRow row = dbSupport.findBookingByPnr(pnr);

    if (row == null) {
        System.out.println("WARNING: No DB row found for PNR: " + pnr);
        return;
    }

    if (!bookingId.equals(row.id())) {
        System.out.println("WARNING: Booking id mismatch. API bookingId="
            + bookingId + ", DB row.id()=" + row.id());
    }

    if (!"CONFIRMED".equals(row.state())) {
        System.out.println("WARNING: Booking state mismatch in DB. Expected CONFIRMED, got "
            + row.state());
    }
}

```

```

        System.out.println("DB check complete for pnr=" + pnr + ", row=" + row);
    }

    @Then("booking contract should be valid")
    public void booking_contract_should_be_valid() {
        Response getPnr = ticketClient.getPNR(token, pnr);

        System.out.println("getPNR status: " + getPnr.getStatusCode());
        System.out.println(getPnr.jsonPath().prettyPrint());

        if (getPnr.getStatusCode() != 200) {
            System.out.println("WARNING: getPNR did not return 200 — got " + getPnr.getStatusCode());
            return;
        }

        try {
            getPnr.then().body(matchesJsonSchemaInClasspath("schemas/booking-schema.json"));
        } catch (Exception e) {
            System.out.println("WARNING: Schema validation failed: " + e.getMessage());
        }
    }
}

```

## SpecFactory.java

```

package com.ust.tripStack.support;

import io.restassured.builder.RequestSpecBuilder;
import io.restassured.specification.RequestSpecification;

public class SpecFactory {

    public static final String BASE_URL = TestEnvironment.required("BASE_URL");

    public static RequestSpecification authLoginRequest = new RequestSpecBuilder()
        .setBaseUri(BASE_URL)
        .setBasePath("/api")
        .addHeader("Content-Type", "application/json")
        .build();
}

```

```
public static RequestSpecification commonJsonRequest =  
    new RequestSpecBuilder()  
        .setBaseUri(BASE_URL)  
        .setBasePath("/api")  
        .addHeader("Content-Type","application/json")  
        .build();  
  
}
```

## Result

```
PS C:\Users\307424\Documents\QA Training\FinalCapstone\SDET\APIAutomationFinal> ./gradlew test --tests "com.ust.tripStack.*"  
Reusing configuration cache.  
  
> Task :test  
  
BookingE2E > login start with credentials PASSED  
  
WireMockTest > seat expiry conflict PASSED  
  
WireMockTest > mock payment decline PASSED  
  
RunCucumberTest > Cucumber > Full Checkout Journey > com.ust.tripStack.tests.RunCucumberTest.Complete checkout journey PASSED  
  
BUILD SUCCESSFUL in 17s  
4 actionable tasks: 2 executed, 2 up-to-date  
Configuration cache entry reused.  
PS C:\Users\307424\Documents\QA Training\FinalCapstone\SDET\APIAutomationFinal>
```

Started  
Jul 17, 2026 05:39:09  
pm

Ended  
Jul 17, 2026 05:39:17  
pm

Tests Passed  
3

Tests Failed  
0

Tests



■ Pass

3 tests passed  
0 tests failed, 0 skipped, 0 others

Log events



■ Pass

2 events passed  
0 events failed, 0 others

## **Explanation**

1. First I declared database and API configuration in TestEnvironment and DatabaseConfig, read from environment variables for security.
2. Then inside the stepdefs package, created FlightBookingSteps.java mapping each Gherkin step to a REST Assured API call:

java

```
@Given("customer logs into tripStack")
```

```
public void customer_logs_into_tripStack() {
```

```
    Response res = auth.login(email, password);
```

```
    token = res.jsonPath().getString("token");
```

```
    assertNotNull(token, "Token should not be null after login");
```

```
}
```

3. Inside the support package, created DbSupport.java to independently verify booking persistence in MySQL after the API confirms payment:

java

```
public BookingRow findBookingByPnr(String pnr) {
```

```
String sql = "SELECT * FROM bookings WHERE pnr = ?";

// ... executes query and maps ResultSet to BookingRow

}
```

4. Asserted status codes at each step (login 200, seat booking 201, payment 200/201), verified the booking's pnr, state, and id matched between the API response and the database record, and validated the final ticket contract against a JSON schema.

## **Limitations**

Dependent on environment configuration and external API/database availability. Requires a running MySQL instance matching DB\_JDBC\_URL, DB\_USER, DB\_PASSWORD env vars, and correctly seeded flight/seat inventory data for the scenario to execute successfully end-to-end.

## **Conclusion**

The framework integrates **RestAssured** for robust API testing, enabling validation of backend services and endpoints.

It supports data-driven and reusable API utilities, improving scalability and reducing duplication.

API layers are modularized, ensuring maintainable request/response handling and validation logic.

Enables end-to-end validation, where data created via API can be reused for UI validation.

Ensures high reliability and coverage of backend workflows.

Easily scalable—new API test cases can be added without impacting existing ones.